

Oefeningen

Kleurwissel bij hover

Normaal wordt CSS gebruikt om de achtergrondkleur van een element te veranderen wanneer er met de muis overheen gegaan wordt (`:hover` -pseudoklasse).

In deze oefening wordt dit niet met CSS, maar met JavaScript gedaan.

events: `mouseover` en `mouseout`

Maak een HTML-pagina met een `div` -element van 200x200 pixels met een achtergrondkleur naar keuze (bv. `rebeccapurple`).

Wanneer de muis over het `div` -element beweegt, moet de achtergrondkleur veranderen in een andere kleur (bv. `deeppink`).

Wanneer de muis het `div` -element verlaat, moet de achtergrondkleur terugkeren naar de oorspronkelijke kleur.

▼ Hint

Je kunt twee event-listeners toevoegen aan het `div` -element: één voor het `mouseover` -event en één voor het `mouseout` -event.

```
const box = document.querySelector("#box");

box.addEventListener("mouseover", () => {
  // verander de achtergrondkleur
```

js

```
});

box.addEventListener("mouseout", () => {
  // zet de achtergrondkleur terug
});
```

▼ Oplossing

```
<div></div>
```

html

```
div {
  width: 200px;
  height: 200px;
  background-color: rebeccapurple;
}
```

CSS

```
const div = document.querySelector("div");

div.addEventListener("mouseover", () => {
  div.style.backgroundColor = "deeppink";
});

div.addEventListener("mouseout", () => {
  div.style.backgroundColor = "rebeccapurple";
});
```

js



Spiegelschrift

Maak een HTML-pagina met een tekst invoerveld (`input type="text"`) en een leeg `p`-element eronder.

Wanneer de gebruiker tekst typt in het invoerveld, moet de tekst in omgekeerde volgorde verschijnen in het `p`-element (spiegelschrift).

event: `input`

▼ Hint

Je kunt een event-listener toevoegen aan het invoerveld die luistert naar het `input`-event. In de event-handler kun je de waarde van het invoerveld ophalen, deze omkeren via JavaScript en vervolgens instellen als de tekstinhoud van het `p`-element.

```
const input = document.querySelector("input");

input.addEventListener("input", (event) => {
  const value = event.target.value; // Haal de huidige waarde
  van het invoerveld op
  // Voeg hier de code toe om de tekst om te keren
  // Voeg hier de code toe om de omgekeerde tekst in het p-
```

js

```
element te plaatsen  
});
```

▼ Hint

Omkeren van een string in JavaScript kan gedaan worden door de string om te zetten naar een array (met `split('')`), de array om te keren (met `reverse()`) en vervolgens weer samen te voegen tot een string (met `join('')`).

▼ Oplossing

```
<input type="text">  
<p></p>
```

html

```
const input = document.querySelector("input");  
const p = document.querySelector("p");  
  
input.addEventListener("input", (event) => {  
  const value = event.target.value;  
  p.textContent = value.split("").reverse().join("");  
});
```

js

Keylogger

Maak een HTML-pagina aan en luister op heel de pagina naar het `keydown`-event. Toon op de pagina welke toets de gebruiker heeft

ingedrukt. Plaats geen element in de `body`, maak alle elementen aan via JavaScript.

Toon de tekst in een `p`-element dat via JavaScript wordt aangemaakt en toegevoegd aan de `body`.

Voorzie een `css`-bestand waarin je zorgt dat de tekst van het `p`-element:

- in het midden van de pagina getoond wordt
- een heel groot lettertype heeft (bv. 100px)

event: `keydown`

▼ Hint

Je kunt een event-listener toevoegen aan het `document` object die luistert naar het `keydown`-event. In de event-handler kun je de eigenschap `key` van het event-object gebruiken om te bepalen welke toets is ingedrukt.

```
document.addEventListener("keydown", (event) => {  
  const pressedKey = event.key; // Haal de ingedrukte toets  
  op  
  // Voeg hier de code toe om de ingedrukte toets op de  
  pagina weer te geven  
});
```

js

▼ Oplossing

```
* {  
  box-sizing: border-box;  
  padding: 0;  
  margin: 0;  
}  
  
body {  
  display: flex;  
  justify-content: center;
```

CSS

```
}

p {
  font-size: 100px;
}
```

```
const p = document.createElement("p");
document.body.appendChild(p);

document.addEventListener("keydown", (event) => {
  p.textContent = event.key;
});
```

js

Extra uitdaging

Hou in een object bij hoe vaak elke toets is ingedrukt. Toon de inhoud van dit object door middel van `JSON.stringify()` op de pagina onder de laatst ingedrukte toets.

y

`{"b":1,"a":1,"r":2,"y":1}`

Uitbreiding:

- Zorg ervoor dat het niet hoofdlettergevoelig is, dus 'a' en 'A' worden als dezelfde toets beschouwd.
- Toon onder de `JSON.stringify()` een tabel met daarin per rij een toets en hoe vaak deze is ingedrukt. Bouw deze tabel op via JavaScript.
 - De hoofding van de tabel bevat de kolomtitels "Toets" en "Aantal keer ingedrukt".
 - Elke volgende rij bevat een toets en het aantal keer dat deze is ingedrukt.
 - Zorg via CSS dat de oneven rijen een andere achtergrondkleur hebben dan de even rijen.
 - Zorg via CSS dat er een rand rond de tabel en de cellen staat met border-collapse op `collapse`.
- Er hoeft geen rekening gehouden te worden met `thead` en `tbody`, enkel met `table` en `tr` / `td` / `th`.

y

{"b":1,"a":1,"r":2,"y":1}

Toets	Aantal keer ingedrukt
b	1
a	1
r	2
y	1

▼ Oplossing

CSS

```
* {
  box-sizing: border-box;
  padding: 0;
  margin: 0;
}

body {
  display: flex;
  align-items: center;

  flex-direction: column;
}

p {
  font-size: 100px;
}

table {
  border-collapse: collapse;
}
```



```
th, td {  
  border: 1px solid black;  
}
```

```
const p = document.createElement("p");  
document.body.appendChild(p);  
  
const jsonOutput = document.createElement("div");  
document.body.appendChild(jsonOutput);  
  
const keys = {};  
  
document.addEventListener("keydown", (event) => {  
  p.textContent = event.key;  
  
  const key = event.key.toLowerCase();  
  if (keys[key] !== undefined) {  
    keys[key] = keys[key] + 1;  
  } else {  
    keys[key] = 1;  
  }  
  
  jsonOutput.textContent = JSON.stringify(keys);  
  
  showKeysInTable();  
});  
  
function showKeysInTable() {  
  let table = document.querySelector("table");  
  
  if (!table) {  
    table = document.createElement("table");  
  }  
  
  table.textContent = ""; // Tabel leegmaken zodat deze vanaf  
  0 terug kan worden opgebouwd  
  
  const trHead = document.createElement("tr");  
  table.appendChild(trHead);  
  
  const thKey = document.createElement("th");  
  thKey.textContent = "Toets";
```

js

```
trHead.appendChild(thKey);

const thAmount = document.createElement("th");
thAmount.textContent = "Aantal keer ingedrukt";
trHead.appendChild(thAmount);

for (const key in keys) {
  const trKey = document.createElement("tr");
  table.appendChild(trKey);

  const tdKey = document.createElement("td");
  tdKey.textContent = key;
  trKey.appendChild(tdKey);

  const tdAmount = document.createElement("td");
  tdAmount.textContent = keys[key];
  trKey.appendChild(tdAmount);
}

document.body.appendChild(table);
}
```

Afbeelding verplaatsen

Maak een HTML-pagina aan met een afbeelding van een Pokémon (of een andere afbeelding naar keuze). Zorg ervoor dat de afbeelding initieel in de linkerbovenhoek van de pagina staat.

Wanneer de gebruiker op de pijltjestoetsen drukt (omhoog, omlaag, links, rechts), moet de afbeelding 10 pixels verplaatsen in de overeenkomstige richting.

Stel de `position`-eigenschap op `absolute` in zodat je de `top` en `left` eigenschappen kunt aanpassen om de afbeelding te verplaatsen.

Pijltoetsen:

- Omhoog: wijzig de `top`-eigenschap met -10 pixels
- Omlaag: wijzig de `top`-eigenschap met +10 pixels
- Rechts: wijzig de `left`-eigenschap met +10 pixels
- Links: wijzig de `left`-eigenschap met -10 pixels

event: `keydown`

▼ Hint

Je kunt een event-listener toevoegen aan het `document` object die luistert naar het `keydown`-event. In de event-handler kun je de eigenschap `key` van het event-object gebruiken om te bepalen welke toets is ingedrukt. Afhankelijk van de ingedrukte toets, kun je de `top` en `left` CSS-eigenschappen van de afbeelding aanpassen om deze te verplaatsen.

Doe een `console.log` van het event om te zien welke waarde `event.key` heeft voor de pijltjestoetsen.

```
document.addEventListener("keydown", (event) => {  
  const key = event.key;  
  console.log(key); // Bekijk welke toets is ingedrukt
```

js

```
// Voeg hier de code toe om de afbeelding te verplaatsen  
});
```

▼ Hint

Om de afbeelding te verplaatsen, kun je de huidige `top` en `left` waarden van de afbeelding ophalen, deze aanpassen op basis van de ingedrukte toets, en vervolgens de nieuwe waarden instellen. Zorg ervoor dat je de waarden als pixels (`px`) instelt.

```
const img = document.querySelector("img");  
  
img.style.top = "10px";  
  
// Haal de huidige top waarde op (wanneer deze nog niet is  
// ingesteld, gebruik 0)  
const currentTop = parseInt(img.style.top) || 0;  
const newTop = currentTop + 10;  
  
// De top-eigenschap moet een string zijn met "px" erachter  
img.style.top = newTop + "px";
```

js

▼ Oplossing

```

```

html

```
img {  
  position: absolute;  
  top: 0;  
  left: 0;  
}
```

CSS

js

```
const img = document.querySelector("img");

document.addEventListener("keydown", (event) => {
  const key = event.key;

  let top = 0;
  if (img.style.top) {
    top = parseInt(img.style.top);
  }

  let left = parseInt(img.style.left) || 0;

  if (key === "ArrowUp") {
    top = top - 10;
  } else if (key === "ArrowDown") {
    top = top + 10;
  } else if (key === "ArrowRight") {
    left = left + 10;
  } else if (key === "ArrowLeft") {
    left = left - 10;
  }

  img.style.top = top + "px";
  img.style.left = left + "px";
});
```



Extra uitdaging

- Toon de huidige coördinaten (top en left) van de afbeelding ergens op de pagina en werk deze bij telkens wanneer de afbeelding verplaatst wordt. Je mag het element hiervoor al in de HTML plaatsen om deze vervolgens via JavaScript bij te werken.
- Wanneer er op de afbeelding geklikt wordt, moet deze terugkeren naar de oorspronkelijke positie (linkerbovenhoek).
- Zorg ervoor dat buiten de pijltoetsen ook WASD (indien jouw toetsel een Qwerty-toetsenbord heeft) of ZQSD (indien jouw toetsel een Azerty-toetsenbord heeft) gebruikt kan worden om de afbeelding te verplaatsen:
 - W of Z: omhoog
 - S: omlaag
 - D: rechts
 - A of Q: links

▼ Oplossing

html

```
<p class="coordinates">Top: 0px, Left: 0px</p>

```

CSS

```
img {
  position: absolute;
  top: 0;
  left: 0;
}

.coordinates {
  position: absolute;
  top: 0;
  left: 0;
}
```

js

```
const img = document.querySelector("img");

function updateCoordinates(top, left) {
  const element = document.querySelector(".coordinates");
  element.textContent = `Top: ${top}px, Left: ${left}px`;
}

document.addEventListener("keydown", (event) => {
  const key = event.key;

  let top = 0;
  if (img.style.top) {
    top = parseInt(img.style.top);
  }

  let left = parseInt(img.style.left) || 0;

  if (key === "ArrowUp" || key === "w") {
    top = top - 10;
  } else if (key === "ArrowDown" || key === "s") {
    top = top + 10;
  }
});
```

```
    } else if (key === "ArrowRight" || key === "d") {  
      left = left + 10;  
    } else if (key === "ArrowLeft" || key === "a") {  
      left = left - 10;  
    }  
  
    img.style.top = top + "px";  
    img.style.left = left + "px";  
  
    updateCoordinates(top, left);  
  });  
  
  document.addEventListener("click", (event) => {  
    img.style.top = 0;  
    img.style.left = 0;  
    updateCoordinates(0, 0);  
  });
```

T, Left: 0px

Previous page

Theorie

Next page

Theorie